
task-app 30日間カリキュラム

Day 01

開発環境を整えて初めてのアプリを動かそう

Next.js 15 / TypeScript / Prisma / tRPC

目次

今日のゴール	3
なぜこれを作るのか	5
前提.....	6
実装ステップ一覧	15
Step 1: 配布 ZIP を展開した場所から始める(読む目安: 5分)	16
Step 2: scaffold-from-scratch.sh を走らせる(読む目安: 10分)	20
Step 3: npm run dev で初期画面を起動する(読む目安: 3分).....	29
Step 4: 自分専用の最初のページを作る(読む目安: 25分).....	32
Step 5: 仕上げた画面をブラウザで見る(読む目安: 3分)	74
今日手に入れたもの	76
つまずきポイント	76
今日学んだ用語	78
応用課題: 角丸の変更が届く範囲を確かめる	79
理解チェック	80
次回予告	81
次に読むもの	81
奥付	82

このカリキュラムでは30日かけて自分専用のタスク管理アプリを作ります。今日はその1日目です。各日の作業は 学びのロードマップ^{*1} で確認できます。

今回は配布 ZIP を展開した場所から開発環境を立ち上げます。ブラウザに最初の画面を表示するところまで進めます。

今日用意するのはタスク管理ツール Linear の配色を参考にした画面です。

今日のゴール

配布 ZIP を展開した場所で `task-app` の土台を起動して

`http://localhost:3000` に最初の画面を表示します。そのあとで design token(色や角丸などの値に名前を付け、まとめて管理する変数)を整えます。

今日つくるページは2枚です。トップページ(`/`)にはヘッダーと大きなカード、右側の小さなカード3枚が並びます。もう1枚は明日から編集する `/dashboard` です。下のスクリーンショットは `/dashboard` を示しています。

1. https://drive.google.com/file/d/10py_p-6ry4RDlxBSyzPn4rtzZe74ND-T/view?usp=drivesdk

Day 01: 開発環境を整えて初めてのアプリを動かそう



図1: Day 01 の最後に作るダッシュボード画面

スクリーンショット: Day 01 の最後に作る `/dashboard` の画面です。見出しと説明文の配置を確認するための見本です。

- ☐ 配布 ZIP を展開して `README.md` と `scripts` が見える場所を `task-app` の作業場所にします。
- ☐ `scripts/scaffold-from-scratch.sh` を実行して開発に必要なファイルを作ります。
- ☐ `npm run dev` で Next.js(React の画面を動かすための土台一式)の初期画面を表示します。
- ☐ `src/app/globals.css` に Linear 風 design token を入れます。
- ☐ `src/app/page.tsx` を自分専用の最初の画面に置き換えます。
- ☐ `src/app/dashboard/page.tsx` を作って明日編集するページを用意します。

なぜこれを作るのか

初日は開発環境の準備に続けて画面も編集します。ファイルを書き換えた結果をブラウザで確かめる練習です。

今日は `globals.css` と `page.tsx` を編集して配色やレイアウトを変えます。

design token を先に整えるのは後の Day でも同じ色を使うためです。

`bg-primary` で主役の色を指定しておけば Day 05 のログイン画面と Day 09 のプロジェクト一覧にも同じ配色を使えます。

design token は色に名前を付けた変数です。同じ変数を使う部品はその値を変えたと一緒に色が変わります。

新しく学ぶ概念

概念	読み方	役割	例え
React	リアクト	画面の部品(コンポーネント)を組み合わせて UI を作るライブラリ	レゴブロック。小さな部品を組み立てて画面全体を作る
Next.js	ネクストジェイエス	React アプリをすぐ動かせるフレームワーク	React 用の工具セット。ルーティング(URL と表示する画面を結び付ける仕組み)やビルドが最初から入っている
JSX	ジェイエスエックス	JavaScript の中に HTML のように画面を書ける構文	JavaScript の処理と画面の記述を1か所に行ける

概念	読み方	役割	例え
コンポーネント	—	画面の部品を関数で定義したもの。 <code>function Home()</code> のように書く	レゴの1ブロック。組み合わせでページを作る
npm	エヌピーエム	パッケージ(ライブラリ)を管理するツール	アプリの材料を取り寄せる配達サービス
TypeScript	タイプスクリプト	JavaScript に型を足した言語	変数に「数値だけ」などの型を指定する。違う型の値を入れると型検査で指摘される

前提

今日はアプリを編集する前に起動環境を用意します。まず次のツールが必要です。

必須

- Node.js `22.12.0` 以上 `23.0.0` 未満(`22.22.2` 推奨)
- npm `10.x`
- Docker Desktop または Docker Engine(この上で PostgreSQL を動かす環境)
- エディタ(VS Code など)
- ターミナル
- ブラウザ

下の表でツールの役割を確認しましょう。そのあとで必要なものを順番にインストールします。

ツール	役割	入手方法
Node.js	JavaScript をパソコン上で動かす実行環境。Next.js を動かすために必須	公式サイトからインストール(下の手順で説明します)
npm	ライブラリ(他の人が作った部品)を管理するツール	Node.js に同梱されるので個別インストール不要
Docker	PostgreSQL を動かすための「コンテナ」実行環境	OS 別の公式手順からインストール(下の手順で説明します)
PostgreSQL	入力したデータを保存するデータベース	Docker から起動するので個別インストール不要
エディタ	コードを書くアプリ	VS Code ^{*2} を推奨

このカリキュラムは macOS を基準に説明します。Windows は「WSL2」で動く Ubuntu を使います。WSL2 が無い場合は先に Microsoft の公式手順^{*3}に従って WSL2 と Ubuntu を入れてください。以降のコマンドはすべて Ubuntu のターミナルで実行します。Ubuntu 22.04 を直接使っている場合も「Ubuntu の場合」と書かれた手順を選びます。

2. <https://code.visualstudio.com/>

3. <https://learn.microsoft.com/ja-jp/windows/wsl/install>

ターミナルを開く

これから何度も「ターミナル」を使います。ターミナルは「文字でパソコンに命令を出す画面」です。開き方は次のとおりです。

- macOS の場合は `command + スペース` で Spotlight を開く。`ターミナル` と入力して Enter を押す
- Windows (WSL2) の場合はスタートメニューで `Ubuntu` を検索して起動
- Ubuntu の場合は `Ctrl + Alt + T` でターミナルを起動

ターミナルには次の手順に載せたコマンドを1行ずつ入力します。

Ubuntu と Windows の WSL2 では、`sudo` を付けたコマンドが登場します。

`sudo` は管理者の権限でコマンドを実行する指定です。Ubuntu のログインパスワードを求められたら入力して Enter を押します。入力中は文字や `*` が表示されませんが、キー入力は受け付けられています。

VS Code をインストールする

この教材の編集手順では VS Code を使います。すでに起動できる場合は本体のインストールを飛ばせます。Windows で WSL2 を使う方は、下の WSL 拡張機能も確認してください。

macOS の場合

VS Code の公式 macOS 手順^{*4}を開き、ページ内のダウンロードリンクから macOS 用のインストーラを入手します。

4. <https://code.visualstudio.com/docs/setup/mac>

ダウンロードした `.dmg` を開き、`Visual Studio Code.app` を `Applications` (アプリケーション)ヘドラッグします。

`command + スペース` を押し、`Visual Studio Code` を検索して起動します。
VS Code の画面が開けば準備完了です。

Windows (WSL2) の場合

VS Code 本体は Windows に入れ、WSL 拡張機能(Ubuntu 内のファイルを編集するための追加機能)で Ubuntu につなぎます。

VS Code の公式 Windows 手順^{*5}から Windows 用の User Installer をダウンロードして実行します。

インストーラの追加設定で `Add to PATH` を選びます。PATH はコマンドを探す場所の一覧です。インストール後は Ubuntu のターミナルを開き直します。

Windows で VS Code を起動します。左側の四角が並んだ拡張機能アイコンを押し、`WSL` を検索します。Microsoft が提供する WSL 拡張機能をインストールします。

Ubuntu 内のプロジェクトを開く操作は Step 4 で行います。VS Code の公式 WSL 手順^{*6}にも、この接続方法が載っています。

Ubuntu 22.04 を直接使っている場合

Ubuntu のターミナルで、次を1行ずつ実行します。

5. <https://code.visualstudio.com/docs/setup/windows>

6. <https://code.visualstudio.com/docs/remote/wsl>

Day 01: 開発環境を整えて初めてのアプリを動かそう

```
sudo snap install --classic code
code --version
```

1行目は Snap(Ubuntu のアプリ配布・更新の仕組み)から VS Code を入れます。2行目でバージョン番号が表示されればインストールできています。VS Code の公式 Linux 手順^{*7}で配布方法を確認できます。

`snap: command not found` と出る環境では、まず Ubuntu 向けの Snap 導入手順^{*8}を確認します。導入後はログアウトして入り直し、上の2行を実行してください。

Node.js をインストールする

この教材で使うのは Node.js 22.12.0 以上 23 未満と npm 10.x です。検証に使う推奨版は Node.js 22.22.2 です。`node -v` がこの範囲、`npm -v` が

10. で始まる場合だけインストール手順を飛ばせます。テスト実行にも使う開発用ツールの Vite 7 は Node.js 22.12.0 以上を対応範囲としているため、次の手順で教材とそろえます。

macOS の場合

1. Node.js 22.22.2 の公式配布一覧^{*9}を開きます。
2. `node-v22.22.2.pkg` をクリックしてダウンロードします。
3. ダウンロードした `.pkg` をダブルクリックします。案内に従って「続ける」を押します。

7. <https://code.visualstudio.com/docs/setup/linux>

8. <https://snapcraft.io/docs/tutorials/install-the-daemon/ubuntu/>

9. <https://nodejs.org/dist/v22.22.2/>

4. インストール後にターミナルをいったん閉じて開き直します。

Windows (WSL2) / Ubuntu の場合

Ubuntu のターミナルで次を1行ずつ実行します。

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -  
sudo apt-get install -y nodejs
```

1行目は Node.js 22 の配布元を登録します。2行目は本体をインストールします。

Docker をインストールする

PostgreSQL(データベース)は Docker の上で動かします。Docker 本体が無い場合は先にインストールします。

macOS / Windows (WSL2) の場合

1. Docker Desktop 公式サイト^{*10} を開きます。自分の OS 用のインストーラをダウンロードします。
2. 案内に従ってインストールします。終了後に Docker Desktop を起動します。
3. 画面の隅にクジラのアイコンが表示されたら準備完了です。macOS は上部のメニューバー、Windows は右下の通知領域に表示されます。

Ubuntu の場合

10. <https://www.docker.com/products/docker-desktop/>

Day 01: 開発環境を整えて初めてのアプリを動かそう

1. Docker 公式の Ubuntu 手順^{*11}を開きます。
2. `Install using the apt repository` の順に Docker Engine を入れます。
3. `docker-ce` と一緒に `docker-compose-plugin` も入れます。
4. `docker version` と `docker compose version` が表示されることを確認します。

macOS と Windows では Docker Desktop を起動したままにします。
Ubuntu では Docker Engine が起動していることを
`sudo systemctl status docker` で確認します。

PostgreSQL はどういう状態なら OK か

PostgreSQL は Docker の上で自動的に立ち上がります。パソコンへ個別にインストールする必要はありません。

11. <https://docs.docker.com/engine/install/ubuntu/>

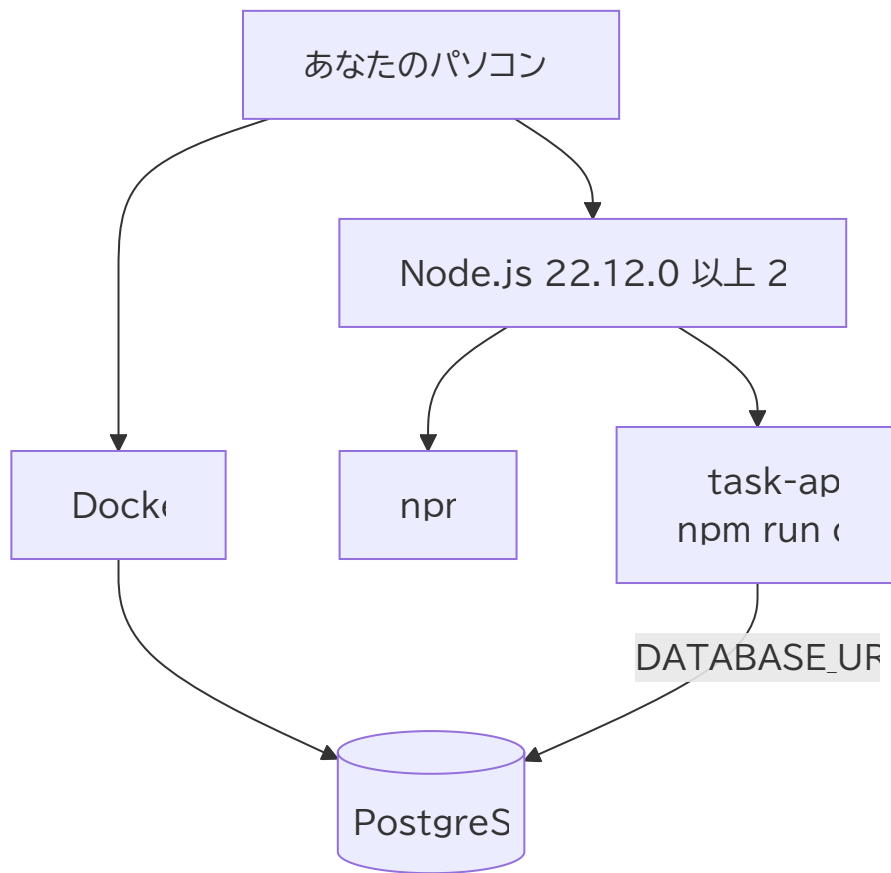


図2: PostgreSQL はどういう状態なら OK か

アプリの実行には Node.js と Docker を使います。npm は Node.js に付いてきます。PostgreSQL は Docker の中で動かします。Docker が止まっているとアプリがデータベースにつなげないため `docker info` で起動状態を先に確かめます。

先に確認しておくコマンド

`Node.js` と `npm` のバージョンはここで見ておきましょう。バージョンが教材の指定と違う場合は先にそろえます。

ターミナル(どこでも OK)

Day 01: 開発環境を整えて初めてのアプリを動かそう

```
node -v  
npm -v
```

この2つはバージョンを表示するコマンドです。`command not found` と出た場合は Node.js がインストールされているか確かめます。インストール直後ならターミナルを開き直して再実行します。

期待する結果

- `node -v` が `v22.12.0` 以上 `v23.0.0` 未満(推奨は `v22.22.2`)
- `npm -v` が `10.x.x` (先頭が `10.`)

Docker が動いているか確認する

PostgreSQL は Docker 上で起動します。次のコマンドで Docker が動いているかを確認しておきましょう。`docker ok` と表示されれば準備完了です。

ターミナル(どこでもOK)

```
docker info >/dev/null 2>&1 && echo "docker ok"
```

このコマンドは Docker の状態を確認します。正常なら `docker ok` と表示します。`>/dev/null 2>&1` は詳しい出力を隠す指定です。`&&` は前のコマンドが成功した場合だけ次を実行します。

何も表示されないときは次を確認してください。

- macOS・Windows: Docker Desktop を開きます。起動が完了したらコマンドを再実行します。

- Ubuntu: `sudo systemctl status docker` で Docker の状態を確認します。

Ubuntu で `docker info` に権限エラーが出た場合は Docker 公式の Linux インストール後の手順^{*12}に従ってください。`docker` グループへの追加後はログアウトしてからログインし直します。

それでも表示されない場合は `docker info` を隠し指定なしで実行してください。表示されたエラーメッセージから起動待ち、インストール不完全、権限の問題などを確認できます。

実装ステップ一覧

ステップ	作業内容	読む時間の目安
Step 1	配布 ZIP を展開した場所から始める	5 分
Step 2	scaffold-from-scratch.sh を走らせる	10 分
Step 3	npm run dev で初期画面を起動する	3 分
Step 4	自分専用の最初のページを作る	25 分
Step 5	仕上げた画面をブラウザで見る	3 分

12. <https://docs.docker.com/engine/install/linux-postinstall/>

Day 01: 開発環境を整えて初めてのアプリを動かそう

読む時間の合計(仮): 約46分です。

表と各 Step に記した時間は、説明とコードを読む時間の仮の目安です。実測した値ではありません。コードの入力、動作確認、ダウンロードや起動の待ち時間、調べものには別に時間を取ってください。

Step 4 で入力するコードは20ブロックです。入力の時間は、表の目安に含めていません。内訳は `globals.css` が11個、トップページが8個、ダッシュボードが1個です。

Step 1: 配布 ZIP を展開した場所から始める(読む目安: 5分)

今日は配布 ZIP を展開した作業場所から始めます。スクリプトで開発に必要なファイルを作ります。

完成形を開くだけでは `package.json` に何が入るか、`.env` に何が書かれるかを自分の目で確かめる場面が一度も来ません。土台を作る手順を確認しておくとの後の Day で起動に失敗してもどのファイルが何のためにあるかを自分の手元からたどれます。

作業用ディレクトリを用意して ZIP を展開する

例としてホームディレクトリの `workspace` フォルダに配布 ZIP を展開します。

次のコードの `if` から `fi` までは、`mise` が入っている場合だけ実行するひとまとまりです。1行ずつ入力すると途中で `>` や `then>` などの入力待ちの印が出ます。エラーではないため、印は入力せず、続く行を `fi` まで入力してください。

ターミナル(どこでもOK)

```
mkdir -p ~/workspace
cd ~/workspace
unzip ~/Downloads/task-app-curriculum-v1.1.zip
cd task-app
if command -v mise >/dev/null 2>&1; then
    mise trust
    mise install
fi
pwd
```

コマンドの役割も確認しておきましょう。

- `mkdir -p ~/workspace`: `workspace` フォルダを作ります(`mkdir` は make directory の略。`-p` を付けると同じ名前のフォルダがすでにあってもエラーになりません)
- `cd ~/workspace`: 作ったフォルダに移動します(`cd` は change directory の略)
- `unzip ...`: 配布 ZIP を展開する
- `cd task-app`: 展開してできた `task-app` フォルダに移動する
- `mise trust` と `mise install`: mise を使う場合だけ配布物の `.mise.toml` に書かれた Node.js を許可してインストールする
- `pwd`: いま自分がどのフォルダにいるかを表示します(`pwd` は print working directory の略)

Windows の WSL2(Ubuntu)では `~/Downloads` は Windows のダウンロードフォルダではありません。Windows 側に保存した ZIP を使うため次のように書き換えてください。

Day 01: 開発環境を整えて初めてのアプリを動かそう

```
unzip "/mnt/c/Users/<Windowsのユーザー名>/Downloads/task-app-curriculum-v1.1.zip"
```

`unzip: command not found` と出たら先に

`sudo apt-get update && sudo apt-get install -y unzip` を実行します。

ZIP の名前は手元のファイルと違うことがあります。バージョン番号の違いやブラウザが付けた末尾の (1) を確かめます。`unzip` で `cannot find` と出たら、macOS と直接利用の Ubuntu では `ls ~/Downloads` で実際のファイル名を確認します。WSL2 では

```
ls "/mnt/c/Users/<Windowsのユーザー名>/Downloads"
```

 を使います。

空白や括弧を含むパスは `"` で囲みます。macOS と直接利用の Ubuntu で末尾に (1) が付いた例は、

```
unzip "$HOME/Downloads/task-app-curriculum-v1.1 (1).zip"
```

です。`$HOME` はホームフォルダを表します。WSL2 では、上の `unzip` の引用符を残し、ZIP の名前を手元の名前へ置き換えます。

この教材の配布 ZIP は `task-app` フォルダに展開されます。

`cd task-app` で `No such file or directory` と出たら、

`cd ~/workspace` で作業場所へ戻り、`ls` でフォルダ名を確認します。

`task-app` があれば `cd task-app` で入ります。なければ、配布された教材用 ZIP を取得し直し、展開手順へ戻ります。以降のコマンドは

`~/workspace/task-app` を使うため、別の名前のフォルダで続けないでください。

期待する結果

- `pwd` の結果が `/Users/あなたのユーザー名/workspace/task-app` のような形になっている(`pwd` は先頭からの完全なパスを表示します。先頭部分はパソコンによって変わります)
- Ubuntu を直接使う場合と Windows の WSL2 では `/home/あなたのユーザー名/workspace/task-app` のような形になります。
- `README.md` と `scripts` が見える配布物ルートにいます。

ここで置いておく配布物

この Day は ZIP を展開した直後の配布物ルートで作業している前提で進めます。

見えていてほしい主なファイルとフォルダは以下の通りです。

- `README.md`
- `doc`
- `scripts`
- `scripts/scaffold-from-scratch.sh`
- `.env.example`

`.env.example` はドットで始まるファイルです。 `ls` だけでは表示されません。見たいときは `ls -a` を使ってください。表示されなくても壊れていません。

今いる場所が配布物ルートになっているか確認しておきましょう。

ターミナル(`~/workspace/task-app`)

```
ls
ls scripts/scaffold-from-scratch.sh
```

Day 01: 開発環境を整えて初めてのアプリを動かそう

`ls` はフォルダの中身を一覧にするコマンドです。`README.md` と `scripts` が見えないときは作業場所を確かめます。`task-app` フォルダが見えていれば

`cd task-app` で中に入ります。それ以外の場所にいる場合は

`cd ~/workspace/task-app` で移動してください。

2行目は `scripts` の中に実行するファイルがあるかを確認します。ファイルがあれば `scripts/scaffold-from-scratch.sh` と表示されます。1行目の `ls` だけではフォルダの中までは表示されません。

期待する結果

- `scripts` フォルダが見えています。
- `scripts/scaffold-from-scratch.sh` が見えています。

Step 2: scaffold-from-scratch.sh を走らせる(読む目安: 10分)

ここが Day 01 のいちばん大事なところです。

Next.js アプリは通常 `npx create-next-app` で設定を選びながら作ります。この教材では設定をまとめた `scripts/scaffold-from-scratch.sh` を使います。スクリプトとは命令をまとめたファイルです。実行すると必要なファイルの作成と依存パッケージの追加が進みます。

スクリプトは次の順番で処理します。

1. Node.js のバージョン確認
2. npm のバージョン確認
3. PostgreSQL を使えるか確認
4. `package.json` と `src/app` がまだ無ければ、配布物を自動で一時退避して `create-next-app` を実行

5. このカリキュラムで使う依存パッケージを追加
6. `package.json` と `tsconfig.json` を整え、ESLint 設定を外して Biome 設定を作成
7. 配布物の `.env.example` を確認
8. shadcn/ui(ボタンなどの画面部品集)の部品とアプリ共通の `layout.tsx` などを配置
9. Prisma(データベースを TypeScript から扱うための道具)のスキーマ(データベースにどんな表と項目を作るかを書いた設計図)と Docker Compose を配置
10. `.env` を作成し、DB の所属と接続先を確認してから PostgreSQL の起動、DB へのスキーマ反映、Prisma Client の生成、初期データの投入を順番に実行

設定の作成はスクリプトで行います。初日は作成されたファイルを確認したあとで動く画面を見るところまで進めます。

実行コマンド

次のコマンドでは `bash` にスクリプトを渡します。この実行方法に実行権限は要りません。`bash` はスクリプトの命令を上から順番に実行するプログラムです。

ターミナル(`~/workspace/task-app`)

```
bash scripts/scaffold-from-scratch.sh
```

Day 01: 開発環境を整えて初めてのアプリを動かそう

このスクリプトを使うのは Day 01 の初期セットアップ時だけです。Day 02 以降に実行すると `src/component/` に自分で書いたファイルが配布版で上書きされます。初期データのプロジェクトとその中に自分で作ったタスクやコメントも消えます。問題が起きた場合は先に付録のトラブルシューティングを読んでください。

別のフォルダで作った DB と名前が重なる場合や、接続先が教材用 DB と異なる場合は、スクリプトが既存の DB を変更せずにエラー終了します。既存の DB は起動状態とデータを保ったままにします。付録のトラブルシューティング^{*13}を開き、「別フォルダの DB と衝突した場合」で今回の名前とポートを分けてから再実行してください。

実行すると `package.json` や `src` が作られます。データベース用のコンテナも起動します。部品をダウンロードするため数分かかることがあります。

`Permission denied` が出たら実行したコマンドを確認してください。`./scripts/...` の形で直接実行する場合は実行権限が必要です。

`bash scripts/scaffold-from-scratch.sh` なら実行権限は不要です。それでも出る場合は作業場所への書き込み権限を確かめます。`pwd` で `~/workspace/task-app` にいるか確認してください。

`Cannot connect to the Docker daemon` が出たら Docker Desktop を起動してから実行し直します。

期待される出力

実行中はターミナルに進行状況が表示されます。内容や順番は環境によって変わります。次は出力例です。

13. <https://drive.google.com/file/d/1OyjFWiGvZBm9QKcnKiougPZidjsn7kqR/view?usp=drivesdk>

Day 01: 開発環境を整えて初めてのアプリを動かそう

下のログは一部を省略した例です。`added ... packages` の数字や秒数は環境によって変わります。スクリプトの出力なので自分で入力する必要はありません。

ターミナル出力(`~/workspace/task-app`)

Day 01: 開発環境を整えて初めてのアプリを動かそう

教材用の初期土台を `/Users/you/workspace/task-app` に作成します。

```
Creating a new Next.js app in
/Users/you/workspace/task-app.
```

```
Using npm.
```

```
Initializing project with template: app-tw
```

```
Installing dependencies:
```

- next
- react
- react-dom

```
Installing devDependencies:
```

- @tailwindcss/postcss
- tailwindcss
- typescript

```
Initialized a git repository.
```

Biome 設定を作成しました。

shadcn/ui コンポーネントを `src/component/ui/` にコピーしました。

Prisma スキーマを配置しました。

`docker-compose.yml` を配置しました。

`.env.example` を `.env` にコピーしました。

Docker で PostgreSQL を起動しています...

(ログはこの後も続きます。自分で入力せず表示を待ちます。)

ここまでの行は「Next.js の土台作成」「必要な部品の取得」「教材用の設定配置」を示します。`Installing dependencies:` は部品のダウンロード中です。

`Biome 設定を作成しました。` が出ると設定ファイルの配置に進んでいます。

```
Prisma スキーマをDBに反映しています...  
シードデータを投入しています...  
DB セットアップが完了しました。  
  
初期セットアップは完了しました。  
カリキュラムの Day 01 の続きを進めてください。
```

最後の `初期セットアップは完了しました。` が完了の目印です。それより前に DB 起動をスキップした案内やエラーがないかも確認します。

`Docker で PostgreSQL を起動しています...` から進まない場合は Docker Desktop の起動状態を確かめます。まだ教材のコードを編集していない初期セットアップ中なら再実行してください。自分で書いたコードや初期プロジェクト内のデータが上書き・削除されるため Day 02 以降は実行しません。

`npm install` の途中で `vulnerabilities` (取得した部品の脆弱性)の件数を表示する場合があります。件数の表示だけでは、インストールの失敗とは判断できません。上の完了の目印と、それまでにエラーが出ていないかを確認します。

`npm audit fix --force` は実行しないでください。教材で使う主要バージョンも更新の対象になります。後のコードを動かせない場合があります。警告の内容は `npm audit` で確認できます。公開するアプリで使い続けてよいかは、指摘された部品と修正版の有無を確認して判断します。

Day 01: 開発環境を整えて初めてのアプリを動かそう

成功判定

次のファイルが見えていれば成功です。

- `package.json`
- `tsconfig.json`
- `biome.json`
- `.env.example`
- `.env`
- `docker-compose.yml`
- `prisma/schema.prisma`
- `src/app/layout.tsx`
- `src/app/page.tsx`
- `src/app/globals.css`
- `src/component/ui/button.tsx`

`.env.example`

このスクリプトは配布物の `.env.example` を `.env` にコピーします。下の見本は今日使う行だけを抜き出しています。実ファイルには `_DEVELOPER_EMAIL` (今日は空欄のまま) や本番用 URL など、この見本に無い設定もあります。これらは後の Day で扱います。

この先はコードの先頭に `filepath:` を含む行が出てきます。これはコードを書くファイルを示す目印です。この教材のための表示です。書き写さなくて構いません。

目印の書き方は置く場所によって3つに分かれます。

書き方	出てくる場所
<code># filepath:</code>	設定ファイルやターミナルのコード
<code>// filepath:</code>	ふつうのコードの中と、開始タグの <code><</code> と <code>></code> のあいだ(属性を書く場所)
<code>{/* filepath: */}</code>	<code>></code> を閉じてから次のタグまでのあいだ

3つともその言語でコメントとして扱われる書き方です。コメントは画面に表示されず処理にも影響しません。

画面の見た目を書く部分(JSX)が2つに分かれるのは場所によってコメントの書き方が変わるためです。

開始タグの `<` と `>` のあいだは属性を書く場所です。ここに置いた `//` はコメントとして読み飛ばされます。

`>` を閉じてから次のタグまでのあいだは画面に文字が出る場所です。この場所の `//` はコメントとして扱われません。そのまま画面に表示されます。タグの設定を書く場所へ `{/* */}` を置くとエラーになって画面が出なくなります。

目印の行は書き写さずに進めても構いません。

Day 01: 開発環境を整えて初めてのアプリを動かそう

```
# filepath: .env.example

_DOCKER_COMPOSE_HOST_PORT_DB=25532

_DOCKER_COMPOSE_HOST_PORT_TEST_DB=25533

DATABASE_URL="postgresql://user:password@localhost:25532/taskapp"
TEST_DATABASE_URL="postgresql://user:password@localhost:25533/taskapp_test"

JWT_SECRET="your-jwt-secret-key-32-chars-minimum-please-change"

NODE_ENV="development"
```

DATABASE_URL はアプリが使うデータベースの接続先です。**localhost:25532** の数字はポート番号(1台のパソコンの中で通信の入口を区別する番号)です。スクリプトが起動する PostgreSQL と同じ番号にしています。通常はこの値で進めます。付録の手順で別の DB とポートを分けた場合は、自分の **.env** に設定した番号を使います。後の Day で接続エラーが出た場合もこの値を確認めます。

危ないアンチパターン

JWT_SECRET は JWT(ログイン状態を証明するデータ)の署名に使うシークレットキー(秘密の文字列)です。他人に知られるとログインの偽装に悪用される恐れがあります。本番用のシークレットキーを GitHub などの公開場所に置いてはいけません。

今日は **.env.example** (設定の見本ファイル)を確認します。Step 2 のスクリプトが見本をコピーして **.env** も作成します。手元の練習では配布された設定値を使います。本番用のシークレットキーを用意したら公開対象から除外した **.env** に記入します。

Step 3: npm run dev で初期画面を起動する(読む目安: 3分)

土台ができたら開発サーバーを起動します。ブラウザで画面を確認しましょう。

編集前の画面を確認しておくとも変更後と比較できます。

開発サーバーを起動する

ターミナル(`~/workspace/task-app`)

```
npm run dev
```

このコマンドはコードをブラウザへ届ける開発サーバーを起動します。通常はパソコンの 3000 番ポートを使います。`Missing script: "dev"` が出たら `pwd` で作業場所を確かめます。場所が違う場合は `cd ~/workspace/task-app` で戻って実行し直します。場所が正しい場合は `cat package.json` を実行してください。`scripts` の中に `"dev": "next dev"` があるか確認します。

`Port 3000 is in use` は別のプログラムがそのポートを使用中という意味です。Next.js は空いているポートへ移るため出力された URL を確認します。

Day 01: 開発環境を整えて初めてのアプリを動かそう

期待される出力

ターミナル出力(`~/workspace/task-app`)

```
> task-app@0.1.0 dev
> next dev

▲ Next.js 15.5.24
- Local:      http://localhost:3000
- Network:    http://192.168.55.2:3000
- Environments: .env

✓ Starting...
✓ Ready in 5.5s
```

Network: の数字と **Ready in** の秒数は環境によって変わります。

Local: の行と **Ready** が見えていれば成功です。

Local: が `http://localhost:3001` などになる場合もあります。3000番が使用中なら Next.js が空いている番号へ移るためです。ブラウザでは **Local:** に出た URL を開きます。この先の `http://localhost:3000` も自分の画面に出ている番号に読み替えてください。

開発サーバーを実行しているあいだは次の入力待ち(プロンプト)が戻りません。このターミナルは閉じずに使います。停止するときは **control + C** を押します。別のコマンドを実行する場合は新しいターミナルを開きます。

ブラウザで開く URL

- `http://localhost:3000`

Local: に別の番号が出ていたらその URL を開きます。

何が見えたら OK か

Next.js のロゴと `Get started by editing src/app/page.tsx.` という「page.tsx を編集してね」という案内文が見えれば大丈夫です。(案内文の文言は Next.js のバージョンによって少し変わります。ロゴ入りの案内ページが出ていれば成功です。)

スクリーンショットの見本

初期画面の見本です。



図 3: Next.js の初期画面がブラウザに表示された状態

Day 01: 開発環境を整えて初めてのアプリを動かそう

編集前の画面を確認する

ここでは Node.js・npm・Next.js・Tailwind(色や余白を短いクラス名で指定する CSS の仕組み)を使って初期画面を表示しています。

編集後にエラーが出た場合は直前に変更した箇所から確かめます。保存前後の表示を見比べながら進めましょう。

Step 4: 自分専用の最初のページを作る(読む目安: 25 分)

ここからが今日のいちばん面白いところです。

初期画面は「Next.js を始める人向けの案内」でした。今回は自分専用のタスク管理アプリのはずです。

配色やレイアウトを `task-app` 用に変えます。

今日編集するファイルは3つです。

1. `src/app/globals.css` に design token(色や角丸などの値に名前を付けて、まとめて管理する変数)を入れます。
2. `src/app/page.tsx` を自分専用の最初の画面に置き換えます。
3. `src/app/dashboard/page.tsx` を作り、Day 02 で使うページを用意します。

編集前に VS Code でプロジェクトを開く

macOS / Ubuntu を直接使っている場合

VS Code を起動し、`File` → `Open Folder...` (日本語表示では「ファイル」→「フォルダを開く...」)を選びます。ホームフォルダ内の `workspace`、`task-app` の順で開いてください。

Windows (WSL2) の場合

`npm run dev` の画面は残したまま、スタートメニューからもう1つ Ubuntu のターミナルを開きます。次を1行ずつ実行します。

```
cd ~/workspace/task-app  
code .
```

`.` は今いるフォルダを指します。初回は Ubuntu 側で VS Code を動かす部品のダウンロードが始まるため、完了まで待ちます。VS Code の左下に `WSL: Ubuntu` など、使っている Ubuntu の名前が表示されることを確認してください。

`code: command not found` と出たら、Windows 側で VS Code をインストールしたか、`Add to PATH` を選んだかを確認します。そのあと Ubuntu のターミナルを開き直して上の2行を実行します。

共通の確認

左側のファイル一覧に `src` や `package.json` があれば、編集するフォルダを開けています。`src/app/globals.css` を開く場合は `src`、`app` の順でフォルダをクリックします。その中の `globals.css` を選びます。

次は、セットアップ後の `task-app` を Ubuntu で開いた画面です。左側の Explorer に `src` と `package.json` が表示されています。OS や表示言語が違って、この2つを確認します。

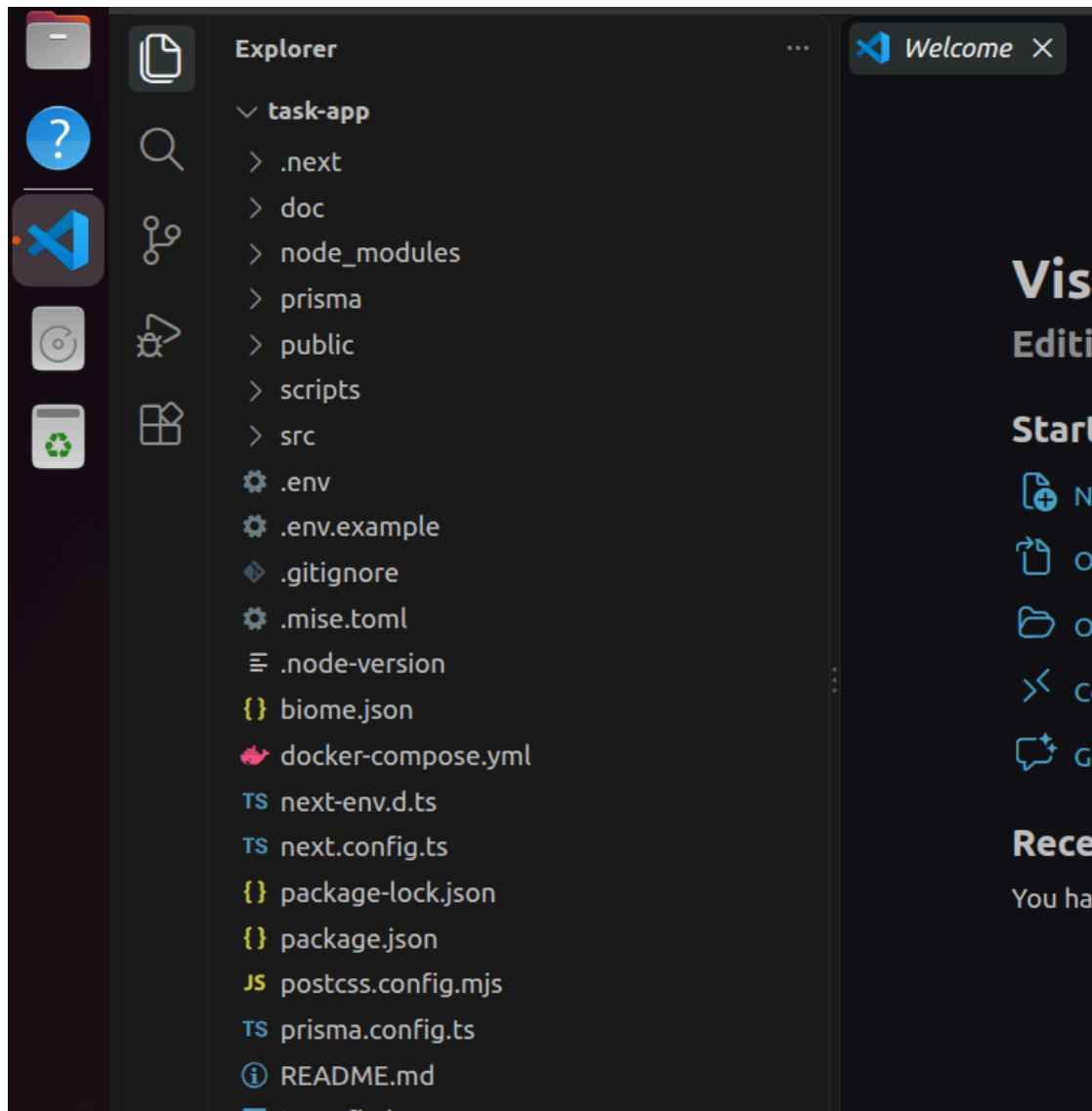


図4: VS Codeでtask-appを開き、左側のExplorerにsrcとpackage.jsonが並んだ状態

初めて開くと **Restricted Mode** (機能を制限した状態)と表示される場合があります。この状態でもファイルの閲覧と編集はできるため、そのまま次へ進めます。

Workspace Trust(フォルダの信頼設定)は、そのフォルダ内のコードを VS Code や拡張機能が実行してよいかを決める設定です。拡張機能を使うため設定を変える場合は、画面上部の **Manage**、または下部の **Restricted Mode** を押します。管理

画面で対象が教材用の `task-app` であることを確認し、配布元と内容を信頼できる場合に `Trust` を選びます。`Restricted Mode` の表示が消えれば設定できています。

親の `workspace` 全体を信頼する必要はありません。判断に迷う場合は制限した状態で編集を続けます。詳しい説明は Workspace Trust の公式手順^{*14}にあります。

`npm run dev` のターミナルは起動したまま使います。ファイルを保存するとブラウザの画面が更新されます。

4-1. `globals.css` を token ベースに差し替える

今の `globals.css` でも画面は表示されます。次は色に役割の名前を付けます。

今日のページでは役割の名前で色を指定します。使う名前は次のとおりです。

名前	役割
<code>bg-primary</code>	主役の色
<code>text-primary-foreground</code>	主役の色の上に乗せる文字色
<code>bg-card</code>	カードの面の色
<code>text-muted-foreground</code>	控えめな説明文の色

こうして役割で名前を付けた色を semantic token(意味を持たせた色の変数)と呼びます。

14. <https://code.visualstudio.com/docs/editing/workspaces/workspace-trust>

Day 01: 開発環境を整えて初めてのアプリを動かそう

役割名で色を指定するには名前と色の対応を決める必要があります。

`globals.css` にその対応を記入します。

編集アンカー

`src/app/globals.css` を開きます。先頭の `@import "tailwindcss";` からファイルの最後まで全部置き換えます。

今回はファイルの一部を修正せず Day 01 は丸ごと入れ替えたほうが理解しやすいです。

このファイルの内容を先に確認する

コードは6つのまとまりに分かれています。先にそれぞれの役割を確認しましょう。

まとまり	書き出し	役割
読み込みと切り替え	<code>@import</code> / <code>@plugin</code> / <code>@custom-variant</code>	Tailwind CSS と拡張プラグインを読み込み、暗いテーマへ切り替える条件を決めます
対応表	<code>@theme inline { ... }</code>	<code>bg-primary</code> などのクラス名と色の変数を結び付けます
アニメーション	<code>@keyframes</code>	折りたたみ部品が開閉するときの動きを決めます
明るいテーマの値	<code>:root { ... }</code>	明るい画面で使う <code>--primary</code> などの値を決めます

まとめり	書き出し	役割
暗いテーマの値	<code>.dark { ... }</code>	暗い画面(ダークモード)で使う変数の値を決めます
ページへの適用	<code>body { ... }</code>	決めた背景色、文字色、フォントをページ全体へ適用します

`hsl(var(--primary))` は変数 `--primary` の値を HSL 形式の色として使う指定です。`:root` の `--primary` を変更するとその変数を使う部品の色が変わります。部品ごとに値を書き換えずに済むよう token を使います。

上から順に貼っていきましょう。各ブロックのあとの「確認ポイント」を確かめて次へ進みます。

Day 01: 開発環境を整えて初めてのアプリを動かそう

```
/* filepath: src/app/globals.css */
@import "tailwindcss" source("../");

@plugin "tailwindcss-animate";

@custom-variant dark (&:is(.dark *));

@theme inline {
  --font-sans:
    var(--font-inter),
    var(--font-noto-sans-jp),
    "Hiragino Kaku Gothic ProN",
    "Hiragino Sans",
    "Meiryo",
    sans-serif;
  --font-mono:
    var(--font-jetbrains-mono),
    "JetBrains Mono",
    "Geist Mono",
    "SFMono-Regular",
    monospace;
```

ここまでで通常の文章とコードに使うフォント候補を登録しました。次のブロックも同じ `@theme inline` の中へ続け、画面の背景やカードで使う色名を変数へ結び付けます。

```
/* filepath: src/app/globals.css(同じファイルの続き)
*/

--color-background: hsl(var(--background));
--color-foreground: hsl(var(--foreground));

--color-card: hsl(var(--card));
--color-card-foreground: hsl(var(--card-foreground));

--color-popover: hsl(var(--popover));
--color-popover-foreground: hsl(
var(--popover-foreground));

--color-primary: hsl(var(--primary));
--color-primary-foreground: hsl(
var(--primary-foreground));

--color-secondary: hsl(var(--secondary));
```

`@import "tailwindcss" source("../");` は Tailwind CSS 本体を読み込み、クラス名を探す場所を `src` フォルダに固定する宣言です。`globals.css` は `src/app` にあるため、`../` が1つ上の `src` を指します。実行ツールが別のフォルダから動いても、`src` 内の `bg-primary` や `min-h-screen` を見落とさないためです。`@plugin "tailwindcss-animate";` は部品の出し入れに使う

Day 01: 開発環境を整えて初めてのアプリを動かそう

`animate-in` や `fade-in` といったクラスを足します。この1行が無いと、それらのアニメーションクラスが働きません。`@custom-variant dark` は `.dark` クラスが付いたときのスタイルを指定する準備です。`@theme inline { ... }` にクラス名と変数の対応を書きま
す。`--color-primary: hsl(var(--primary));` は `bg-primary` など
で使う色を `--primary` に結び付けます。

確認ポイント: `@theme inline {` の中に `--color-` で始まる行が並んでいることを確認します。次のブロックを続けて書きます。

```
--color-secondary-foreground: hsl(
var(--secondary-foreground));

--color-muted: hsl(var(--muted));
--color-muted-foreground: hsl(var(--muted-foreground));

--color-accent: hsl(var(--accent));
--color-accent-foreground: hsl(
var(--accent-foreground));
```

ここまでで控えめな表示と強調表示の色を登録しました。次のブロックでは危険、成功、警告の色を登録してから、枠線・グラフ・サイドバーの色へ続けます。

```
/* filepath: src/app/globals.css(同じファイルの続き)
*/

--color-destructive: hsl(var(--destructive));
--color-destructive-foreground: hsl(
```

```
var(--destructive-foreground));

--color-success: hsl(var(--success));
--color-success-foreground: hsl(
var(--success-foreground));
--color-warning: hsl(var(--warning));
--color-warning-foreground: hsl(
var(--warning-foreground));

--color-border: hsl(var(--border));
--color-input: hsl(var(--input));
--color-ring: hsl(var(--ring));

--color-chart-1: hsl(var(--chart-1));
--color-chart-2: hsl(var(--chart-2));
--color-chart-3: hsl(var(--chart-3));
--color-chart-4: hsl(var(--chart-4));
--color-chart-5: hsl(var(--chart-5));

--color-sidebar: hsl(var(--sidebar));
--color-sidebar-foreground: hsl(
var(--sidebar-foreground));
--color-sidebar-primary: hsl(var(--sidebar-primary));
```

-foreground が付く名前は面の上に乗せる文字色を表します。背景と文字の色をペアで決めておくための命名です。**--color-chart-1** から **--color-chart-5** はグラフ用の色です。**--color-sidebar-** 系はサイドバーで使います。これらは後の Day で呼び出します。

Day 01: 開発環境を整えて初めてのアプリを動かそう

確認ポイント: `--color-sidebar-primary` の行まで書けたことを確かめます。

次のブロックを続けて書きます。

```
--color-sidebar-primary-foreground: hsl(
var(--sidebar-primary-foreground));
--color-sidebar-accent: hsl(var(--sidebar-accent));
--color-sidebar-accent-foreground: hsl(
var(--sidebar-accent-foreground));
--color-sidebar-border: hsl(var(--sidebar-border));
--color-sidebar-ring: hsl(var(--sidebar-ring));

--radius-sm: calc(var(--radius) - 8px);
--radius-md: calc(var(--radius) - 4px);
--radius-lg: var(--radius);
--radius-xl: calc(var(--radius) + 4px);

--shadow-xs: 0 1px 2px 0 rgb(15 23 42 / 0.06);
--shadow-sm: 0 1px 2px 0 rgb(15 23 42 / 0.06), 0 8px
24px -12px rgb(79 70 229 / 0.18);
--shadow-md: 0 2px 4px 0 rgb(15 23 42 / 0.08), 0 18px
44px -20px rgb(79 70 229 / 0.22);
--shadow-lg: 0 8px 24px -12px rgb(15 23 42 / 0.12), 0
28px 64px -28px rgb(79 70 229 / 0.28);

--ease-linear-out: cubic-bezier(0.16, 1, 0.3, 1);
--duration-fast: 120ms;
--duration-base: 180ms;
```

```
--duration-slow: 280ms;  
  
--animate-accordion-down: accordion-down 0.2s ease-out;  
--animate-accordion-up: accordion-up 0.2s ease-out;
```

`--radius-sm` などは基準値 `--radius` から `calc(...)` で計算した角丸です。基準値を変えると計算結果も変わります。`--shadow-` 系は影の強さを表します。`--duration-` 系は動きの速さを決めます。同じ変数を使う部品に共通の値を適用するための設定です。

Day 01: 開発環境を整えて初めてのアプリを動かそう

確認ポイント: `--radius-sm` から `--animate-accordion-up` までを確認します。次のブロックを続けて書きます。

```
@keyframes accordion-down {  
  from {  
    height: 0;  
  }  
  to {  
    height: var(--radix-accordion-content-height);  
  }  
}  
  
@keyframes accordion-up {  
  from {  
    height: var(--radix-accordion-content-height);  
  }  
  to {  
    height: 0;  
  }  
}  
  
@layer base {  
  :root {  
    --background: 0 0% 100%;  
    --foreground: 222 22% 10%;  
  }  
}
```

Day 01: 開発環境を整えて初めてのアプリを動かそう

`@keyframes accordion-down` と `accordion-up` は折りたたみ部品(アコーディオン)の開閉時の動きです。後の Day で使う `shadcn/ui`(ボタンなどの画面部品集)の部品から呼び出します。その下の `}` で `@theme inline` を閉じます。続く `@layer base` と `:root {` は明るいテーマの値を指定する部分です。まず背景色 `--background` と文字色 `--foreground` を決めています。

Day 01: 開発環境を整えて初めてのアプリを動かそう

確認ポイント: `@theme inline` を閉じる `}` とその下の `@layer base` を確認します。次のブロックを続けて書きます。

```
--card: 0 0% 100%;  
--card-foreground: 222 22% 10%;  
  
--popover: 220 33% 99%;  
--popover-foreground: 222 22% 10%;  
  
--primary: 253 77% 60%;  
--primary-foreground: 0 0% 100%;  
  
--secondary: 240 24% 96%;  
--secondary-foreground: 223 20% 16%;  
  
--muted: 225 23% 95%;  
--muted-foreground: 220 11% 42%;  
  
--accent: 191 82% 95%;  
--accent-foreground: 193 73% 24%;  
  
--destructive: 354 70% 54%;  
--destructive-foreground: 0 0% 100%;  
  
--success: 158 64% 41%;  
--success-foreground: 0 0% 100%;
```

`--primary: 253 77% 60%;` の数字は色相・鮮やかさ・明るさの順で色を表す HSL 形式の値です。ここで変数に実際の色を指定します。主役の `--primary` は青紫、危険を知らせる `--destructive` は赤、成功を知らせる `--success` は緑です。

Day 01: 開発環境を整えて初めてのアプリを動かそう

確認ポイント: `--primary` から `--success-foreground` までを確認します。次のブロックを続けて書きます。

```
--warning: 35 92% 55%;
--warning-foreground: 223 20% 12%;

--border: 225 20% 89%;
--input: 225 20% 89%;
--ring: 253 77% 60%;

--radius: 10px;

--chart-1: 253 77% 60%;
--chart-2: 191 72% 42%;
--chart-3: 333 72% 64%;
--chart-4: 35 92% 55%;
--chart-5: 222 18% 48%;

--sidebar: 224 28% 97%;
--sidebar-foreground: 222 22% 12%;
--sidebar-primary: 253 77% 60%;
--sidebar-primary-foreground: 0 0% 100%;
--sidebar-accent: 225 23% 95%;
--sidebar-accent-foreground: 223 20% 16%;
--sidebar-border: 225 20% 89%;
--sidebar-ring: 253 77% 60%;
}
```

`--radius: 10px;` は `calc(...)` の基準にした角丸の値です。変更すると `--radius-sm` などの計算結果にも反映されます。 `--ring` はボタンや入力欄を選んだときに周りへ表示する輪の色です。選択した場所を示すため `--primary` と同じ色にしています。

Day 01: 開発環境を整えて初めてのアプリを動かそう

確認ポイント: `:root { ... }` が `}` で閉じていることを確認します。次のブロックを続けて書きます。

```
.dark {  
  --background: 228 21% 10%;  
  --foreground: 220 20% 97%;  
  
  --card: 228 20% 12%;  
  --card-foreground: 220 20% 97%;  
  
  --popover: 228 20% 13%;  
  --popover-foreground: 220 20% 97%;  
  
  --primary: 254 86% 68%;  
  --primary-foreground: 233 35% 10%;  
  
  --secondary: 226 16% 18%;  
  --secondary-foreground: 220 20% 96%;  
  
  --muted: 226 16% 16%;  
  --muted-foreground: 220 12% 69%;  
  
  --accent: 184 33% 18%;  
  --accent-foreground: 183 85% 84%;  
  
  --destructive: 355 72% 60%;
```

`.dark {` には暗いテーマ(ダークモード)の値を書きます。`:root` と同じ変数名に暗い画面用の値を指定します。ページの外側に `.dark` クラスが付くところの値が優先されます。部品は変数を参照するため部品側のコードを変えずに配色を切り替えられます。

Day 01: 開発環境を整えて初めてのアプリを動かそう

確認ポイント: `.dark {` の中に `:root` と同じ変数名が並んでいることを確かめます。次のブロックを続けて書きます。

```
--destructive-foreground: 0 0% 100%;

--success: 158 60% 46%;
--success-foreground: 0 0% 100%;

--warning: 38 92% 60%;
--warning-foreground: 223 20% 12%;

--border: 224 15% 22%;
--input: 224 15% 22%;
--ring: 254 86% 68%;

--chart-1: 254 86% 68%;
--chart-2: 184 56% 52%;
--chart-3: 330 77% 71%;
--chart-4: 38 92% 60%;
--chart-5: 220 12% 69%;

--sidebar: 228 21% 9%;
--sidebar-foreground: 220 20% 96%;
--sidebar-primary: 254 86% 68%;
--sidebar-primary-foreground: 233 35% 10%;
--sidebar-accent: 226 16% 16%;
--sidebar-accent-foreground: 220 20% 96%;
```

暗いテーマでは `--primary` の明るさを `60%` から `68%` に上げています。暗い背景で主役色を見分けやすくするためです。グラフやサイドバーの色も暗い画面向けの値にしています。

確認ポイント: `--chart-1` から `--sidebar-accent-foreground` までを確認します。次のブロックを続けて書きます。

```
--sidebar-border: 224 15% 22%;
--sidebar-ring: 254 86% 68%;
}

body {
  background-color: hsl(var(--background));
  color: hsl(var(--foreground));
  font-family: var(--font-sans);
  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

button:not(:disabled),
[role="button"]:not(:disabled):not([
aria-disabled="true"]):not([data-disabled]) {
  cursor: pointer;
}
}
```

`body { ... }` は変数をページに適用する部分です。背景色と文字色は HSL 形式で指定しています。フォントも変数で指定しているため変数の値を変えると表示に反映されます。`-webkit-font-smoothing` などは文字の輪郭をなめらかにするブラウザ向けの設定です。有効なボタンへマウスを重ねると、`cursor` の指定で手の形を表示します。押せる要素を見分けるためです。`button` 要素は `disabled` で無効化されていないものに適用します。`button` 以外を `role` 属性でボタンとして扱う場合は、`aria-disabled` が `true` のものや `data-disabled` が付いたものを対象から外します。ここまでで `globals.css` の置き換えは終わりです。

4-2. `page.tsx` を最初の画面に置き換える

次は Next.js の初期画面をタスク管理アプリのトップページに置き換えます。

今日のテーマはこれです。

トップページに見出しとカードを配置する

左側にメインのカードを置きます。右側には機能の予定などを示す小さなカードを並べます。

編集アンカー

`src/app/page.tsx` を開きます。`import Image from "next/image";` からファイルの最後まで全部置き換えます。

初期コンポーネントの `Home` は残さずファイル全体を次の内容に差し替えます。

```
// filepath: src/app/page.tsx
import Link from 'next/link';

export default function HomePage() {
  return (
    <main className="min-h-screen bg-background
text-foreground">
      <div className="mx-auto flex min-h-screen max-w-6xl
flex-col px-6 py-8 lg:px-10">
        <header className="flex flex-col gap-4 rounded-xl
border border-border/80 bg-card/80 px-4 py-4 shadow-sm
backdrop-blur sm:flex-row sm:items-center
sm:justify-between">
          <div className="space-y-1">
            <p className="text-xs font-medium uppercase
tracking-[0.24em] text-muted-foreground">
              タスク管理
            </p>
            <h1 className="text-sm font-semibold
text-card-foreground">
              TaskApp
            </h1>
          </div>

          <div className="inline-flex w-fit items-center
gap-2 rounded-full border border-border bg-secondary px-3
py-1.5 text-xs font-medium text-secondary-foreground">
            <span className="h-2 w-2 rounded-full
bg-primary" />

```

Day 01: 開発環境を整えて初めてのアプリを動かそう

```
        作りかけ
      </div>
    </header>

    <section className="mt-8 grid gap-6
lg:grid-cols-[1.15fr_0.85fr]">
      <div className="overflow-hidden rounded-[28px]
border border-border bg-card shadow-md">
```

`export default function HomePage()` はページに表示する部品の宣言です。Next.js は `src/app/page.tsx` をトップページ(`/`)として使います。`className` の `bg-background` や `text-muted-foreground` は `globals.css` に登録した token を参照します。色を変えたいときは `globals.css` の token の値を書き換えます。

確認ポイント: `export default function HomePage()` と中の `<header>` を確認します。次のブロックを続けて書きます。

```
    {/* filepath: src/app/page.tsx(同じファイルの
    続き) */}

    <div className="border-b border-border px-8
py-6">
      <div className="inline-flex items-center
gap-2 rounded-full bg-accent px-3 py-1 text-sm
font-medium text-accent-foreground">
        タスク管理
      </div>
```

```

        <h2 className="mt-6 max-w-3xl text-4xl
font-semibold tracking-tight text-card-foreground
sm:text-5xl">
            プロジェクトとタスクを、ひとつの画面で。
        </h2>

        <p className="mt-4 max-w-2xl text-base
leading-8 text-muted-foreground">
            担当と期限を決めて、いまやることだけを取り出す。
            まだ作りかけで、画面は入口だけ。
        </p>

        <div className="mt-8 flex flex-col gap-3
sm:flex-row">
            <a
                className="inline-flex items-center
justify-center rounded-lg bg-primary px-5 py-3 text-sm
font-semibold text-primary-foreground shadow-sm
transition-transform duration-200 hover:-translate-y-0.5"
                href="#today-goals"
            >
                これから足すものを見る
            </a>
            <Link
                className="inline-flex items-center
justify-center rounded-lg border border-border
bg-background px-5 py-3 text-sm font-semibold
text-foreground transition-colors hover:bg-secondary"
                href="/dashboard"
            >

```

Day 01: 開発環境を整えて初めてのアプリを動かそう

`<h2>` に見出しを置きます。`<p>` は説明文です。その下にリンクを並べます。

`<a href="#today-goals">` は同じページ内へ移動します。

`<Link href="/dashboard">` は別のページへ移動します。Next.js の `Link` はページ全体を読み込み直さず表示を切り替えるためアプリ内のページ移動に使います。

確認ポイント: `<a>` と `<Link>` の2種類のリンクを確認します。次のブロックを続けて書きます。

```
// filepath: src/app/page.tsx(同じファイルの続き)

    >
      ダッシュボードへ入る
    </Link>
    <a
      className="inline-flex items-center
justify-center rounded-lg border border-border
bg-background px-5 py-3 text-sm font-semibold
text-foreground transition-colors hover:bg-secondary"
      href="#next-step"
    >
      次にやることを見る
    </a>
  </div>
</div>

  <div className="grid gap-4 bg-secondary/60
px-8 py-6 md:grid-cols-3">
    <article className="rounded-2xl border
border-border bg-background px-4 py-4 shadow-xs">
```

```

        <p className="text-xs font-medium
uppercase tracking-[0.18em] text-muted-foreground">
            これから
        </p>

        <p className="mt-3 text-3xl font-semibold
text-foreground">プロジェクト</p>

        <p className="mt-2 text-sm leading-7
text-muted-foreground">
            案件ごとにタスクを分けて置けるようにする。
        </p>
    </article>

    <article className="rounded-2xl border
border-border bg-background px-4 py-4 shadow-xs">

```

大きなカードの下半分には小さなカードを3枚並べます。`md:grid-cols-3` は中くらい以上の画面幅で3列にする指定です。狭い画面では縦に並びます。各カードを `<article>` で囲むのは独立した内容のまとまりを示すためです。色は token 名で指定します。

確認ポイント: 1枚目のカードを閉じたことと2枚目の `<article>` を開いたことを確認します。次のブロックを続けて書きます。

```

        {/* filepath: src/app/page.tsx(同じファイ
ルの続き) */}

        <p className="text-xs font-medium
uppercase tracking-[0.18em] text-muted-foreground">
            これから
        </p>

```

Day 01: 開発環境を整えて初めてのアプリを動かそう

```
        <p className="mt-3 text-3xl font-semibold
text-foreground">タスク</p>

        <p className="mt-2 text-sm leading-7
text-muted-foreground">
            担当と期限を1行で決められるようにする。
        </p>
    </article>

    <article className="rounded-2xl border
border-border bg-background px-4 py-4 shadow-xs">
        <p className="text-xs font-medium
uppercase tracking-[0.18em] text-muted-foreground">
            これから
        </p>
        <p className="mt-3 text-3xl font-semibold
text-foreground">レポート</p>
        <p className="mt-2 text-sm leading-7
text-muted-foreground">
            終わった数と残りを数えられるようにする。
        </p>
    </article>
</div>
</div>

<div className="space-y-4">
```

3枚の小さなカードと左側の大きなカードを閉じました。次は右側の列です。

`id="today-goals"` はこの場所の目印です。リンクの `href="#today-goals"`

と同じ名前にするとクリック時にここまでスクロールします。

確認ポイント: 3 枚目の `<article>` と左側の列を閉じ、右側の列の `<div>` を開いていることを確認します。次のブロックを続けて書きます。

```

    { /* filepath: src/app/page.tsx(同じファイルの
    続き) */}

    <article
      id="today-goals"
      className="rounded-[28px] border
border-border bg-card p-6 shadow-sm"
    >
      <p className="text-sm font-semibold
text-card-foreground">
        これから足すもの
      </p>
      <ul className="mt-4 space-y-3 text-sm
leading-7 text-muted-foreground">
        <li className="rounded-2xl bg-secondary
px-4 py-3">
          プロジェクトを作って、タスクを登録する
        </li>
        <li className="rounded-2xl bg-secondary
px-4 py-3">
          期限と優先度で並べ替える
        </li>
        <li className="rounded-2xl bg-secondary
px-4 py-3">
          進み具合をレポートで見る
        </li>
        <li className="rounded-2xl bg-secondary

```

Day 01: 開発環境を整えて初めてのアプリを動かそう

```
px-4 py-3">  
    メンバーを招いて一緒に使う  
  </li>  
</ul>  
</article>
```

最初のカードはここで閉じています。4項目すべてが `ul` の内側にあり、閉じる順番が `</li>`、`</ul>`、`</article>` になっていることを確認してから、次のカードを開きます。

```
{/* filepath: src/app/page.tsx(同じファイルの  
続き) */}  
  
  <article className="rounded-[28px] border  
border-border bg-card p-6 shadow-sm">  
    <p className="text-sm font-semibold  
text-card-foreground">  
      メモ
```

`<ul>` と `<li>` は箇条書きの HTML タグです。各 `<li>` に `rounded-2xl bg-secondary` を付けて角丸の背景を指定します。日本語の表示文はタグの中に書きます。最後の `メモ` は次のカードの見出しです。まだカードの途中なので次のブロックへ進みます。

確認ポイント: `<ul>` の中に4つの `<li>` があることを確認します。次のブロックを続けて書きます。

```

    { /* filepath: src/app/page.tsx(同じファイル
の続き) */}

    </p>

    <p className="mt-4 text-sm leading-8
text-muted-foreground">
色を変えるときは、globals.css の --primary などの色の変数を変更します。
    同じ変数を使う部品に、変更がまとめて反映されます。

    </p>

    <div className="mt-5 rounded-2xl bg-primary
px-4 py-4 text-primary-foreground shadow-sm">
      <p className="text-xs font-medium
uppercase tracking-[0.18em] text-primary-foreground/80">
        いまの状態
      </p>
      <p className="mt-2 text-lg font-semibold">
        入口の画面が1枚できた
      </p>
    </div>
  </article>

  <article
    id="next-step"
    className="rounded-[28px] border
border-border bg-card p-6 shadow-sm"
  >

```

Day 01: 開発環境を整えて初めてのアプリを動かそう

```
        <p className="text-sm font-semibold
text-card-foreground">
            次にやること
        </p>

        <p className="mt-4 text-sm leading-8
text-muted-foreground">
```

`bg-primary` と `text-primary-foreground` は背景と文字の色をペアで指定しています。末尾の `id="next-step"` はカードの目印です。前に書いた `href="#next-step"` のリンクからこの場所へ移動できます。

確認ポイント: `id="next-step"` の `<article>` を開いたことを確認します。最後のブロックを続けて書きます。

```
{/* filepath: src/app/page.tsx(同じファイルの続き) */}

ダッシュボードを開くと、いまの状況がまとまって見えるようにする。

        </p>
        </article>
    </div>
</section>
</div>
</main>

);
}
```

「次にやること」の本文のあとに `</p>` から `</main>` までの閉じタグを書きます。開いた順とは逆に内側から閉じて `HomePage` の `return` を終わめます。JSX は開いたタグを閉じる必要があります。保存後にエラーが出たら閉じタグの対応を確認します。

4-3. `dashboard/page.tsx` を作る

Day 02 は `src/app/dashboard/page.tsx` に機能を追加します。

準備として Day 01 の最後にダッシュボードのページを用意します。

`src/app` の中に `dashboard` フォルダを作ります。VS Code の左側のファイル一覧で `src/app` を右クリックして「新しいフォルダ」を選びます。名前を `dashboard` にしてください。ターミナルを使う場合は `task-app` で `mkdir src/app/dashboard` を実行します。

作った `dashboard` フォルダを右クリックして「新しいファイル」を選びます。`page.tsx` という名前で次の内容を入力しましょう。

Day 01: 開発環境を整えて初めてのアプリを動かそう

```
// filepath: src/app/dashboard/page.tsx
export default function DashboardPage() {
  return (
    <main className="min-h-screen bg-background
text-foreground">
      <div className="mx-auto flex min-h-screen max-w-5xl
items-center justify-center px-6 py-10">
        <section className="w-full rounded-3xl border
border-border bg-card px-8 py-10 shadow-md">
          <p className="text-sm font-medium uppercase
tracking-[0.24em] text-muted-foreground">
            Dashboard
          </p>
          <h1 className="mt-4 text-4xl font-semibold
tracking-tight text-card-foreground sm:text-5xl">
            ダッシュボード
          </h1>
          <p className="mt-4 max-w-2xl text-base
leading-8 text-muted-foreground">
            まだ表示するデータがありません。
            プロジェクトとタスクを作れるようになると、ここに出ます。
          </p>
        </section>
      </div>
    </main>
  );
}
```

Next.js はフォルダ名を URL の一部として使います。

`src/app/dashboard/page.tsx` に置くと `/dashboard` で表示できます。トップページと同じ token を使うため共通の配色になります。保存したら

`http://localhost:3000/dashboard` を開きます。「ダッシュボード」の見出しを確かめましょう。`404` は、接続したサーバーに指定したページが見つからないときの応答です。まずターミナルに表示された `Local` の URL と、ブラウザのアドレスを比べます。別のアプリが 3000 番ポートを使っていると、今回のアプリは 3001 番などで起動します。今回のアプリの URL に `/dashboard` を付けて開き、その URL でも `404` なら `src/app/dashboard/page.tsx` の置き場所と名前を確認します。接続できないというブラウザの表示は `404` とは別です。その場合は `npm run dev` が動いているかを確認めます。

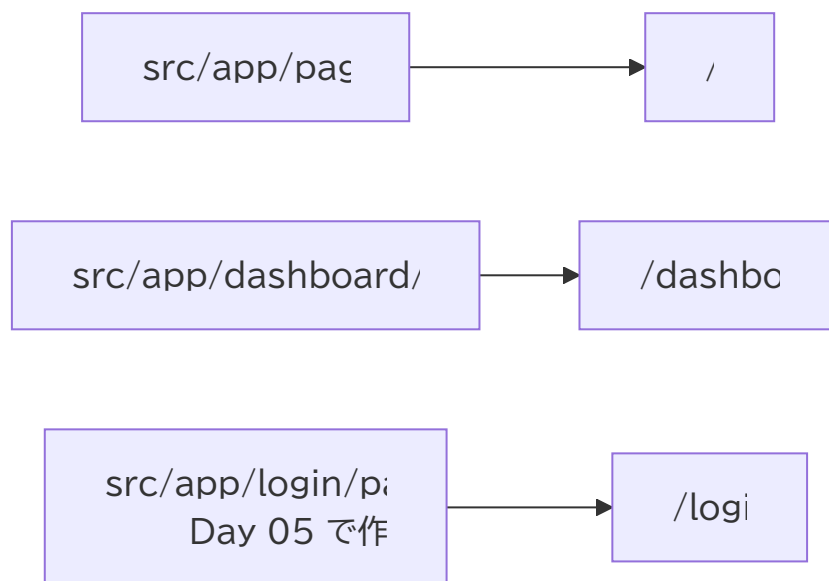


図 5: 4-3. dashboard/page.tsx を作る

フォルダ名が URL に対応します。その中の `page.tsx` が表示内容です。Day 05 のログイン画面と Day 06 の登録画面もこの規則で作ります。`404` が出たら図の左側とファイルの置き場所を見比べます。

Day 01: 開発環境を整えて初めてのアプリを動かそう

ここで押さえないこと

- ルートの `src/app/page.tsx` はトップページ
- `src/app/dashboard/page.tsx` は今後機能を追加するダッシュボード
- Day 02 はダッシュボードに自分のメッセージを追加します。

できあがる見た目のポイント

このページでは今日入れた token をそのまま使っています。

たとえば次の対応です。

- `bg-background` で画面全体の背景
- `bg-card` で主役の面
- `bg-primary` と `text-primary-foreground` で行動を促すボタン(CTA)
- `text-muted-foreground` で説明文
- `border-border` で面同士の境界線

もし色が乗らないとき

だいたいこのどちらかです。

- `src/app/globals.css` の貼り付けが途中で切れています。
- `npm run dev` が止まっている(ターミナルに `Ready` が出ていません)

`globals.css` を開いて `@theme inline` と `:root` の貼り付け漏れを確認しましょう。

Pro パターンで書こう(arbitrary value 多用より design token を先に定義する)

本編の `page.tsx` は役割名で色を指定しています。次は値を直接書く場合との比較です。比較用コードは写経しません。色や余白の指定方法を見比べましょう。

Before(改善前のコード)

```
// filepath: 読み比べ用サンプル(比較用の一部・実ファイルには対応しません)

function WelcomeHero() {

  return (

    <section className="rounded-[28px] border
border-[#25273f] bg-[#0f1021] px-[32px] py-[28px]
shadow-[0_24px_80px_-32px_rgba(99,102,241,0.45)]">

      <span className="inline-flex items-center
rounded-full bg-[#16172d] px-[12px] py-[6px] text-[13px]
font-medium text-[#9aa2c3]">

        タスク管理

      </span>

      <h2 className="mt-[24px] text-[44px] font-semibold
leading-[1.08] tracking-[-0.04em] text-white">

        プロジェクトとタスクを、ひとつの画面で。

      </h2>

      <p className="mt-[18px] max-w-[620px] text-[16px]
leading-[1.9] text-[#b0b7d3]">

        担当と期限を決めて、いまやることだけを取り出す。

        まだ作りかけで、画面は入口だけ。

      </p>

      <div className="mt-[32px] flex gap-[12px]">

        <a

          className="inline-flex items-center
justify-center rounded-[12px] bg-[#6d5dfc] px-[20px]
py-[12px] text-[14px] font-semibold text-white"

          href="#today-goals"

        >
```

Day 01: 開発環境を整えて初めてのアプリを動かそう

```
これから足すものを見る

</a>

<a
  className="inline-flex items-center
  justify-center rounded-[12px] border border-[#2d314b]
  bg-[#151729] px-[20px] py-[12px] text-[14px]
  font-semibold text-white"
  href="#next-step"
>
```

読み比べ用: ここは写経しません。続けてコードを読み進めましょう。

この例は `bg-[#0f1021]` や `text-[13px]` のように値を直接書いています。
`#0f1021` だけでは背景用の色なのかが分かりません。複数の画面で同じ色を使う
と変更時にそれぞれの値を探す必要があります。続きは2つ目のリンクの文言、閉じ
タグ、`HomePage` の宣言です。

```
{/* filepath: 読み比べ用サンプル(続き・実ファイルには対応  
しません) */}  
  
    次にやることを見る  
  
    </a>  
  
  </div>  
  
</section>  
);  
}  
  
export default function HomePage() {  
  return <WelcomeHero />;  
}
```

このコードの問題点:

- 同じ色や角丸を別の画面で使うたびに、値をコピーしなければなりません。
- `#6d5dfc` や `#0f1021` からは色の役割を読み取れません。
- 配色を変えるときに、複数のファイルから指定箇所を探して直さなければなりません。

After(プロが書くコード)

```
// filepath: 読み比べ用サンプル(比較用の一部・実ファイルには対応しません)  
  
function WelcomeHero() {  
  return (  
    <section className="rounded-[28px] border  
border-border bg-card px-8 py-7 shadow-md">  
  
      <span className="inline-flex items-center  
rounded-full bg-accent px-3 py-1 text-sm font-medium
```

Day 01: 開発環境を整えて初めてのアプリを動かそう

```
text-accent-foreground">
    タスク管理
  </span>

  <h2 className="mt-6 text-4xl font-semibold
tracking-tight text-card-foreground sm:text-5xl">
    プロジェクトとタスクを、ひとつの画面で。
  </h2>

  <p className="mt-4 max-w-2xl text-base leading-8
text-muted-foreground">
    担当と期限を決めて、いまやることだけを取り出す。

    まだ作りかけで、画面は入口だけ。
  </p>

  <div className="mt-8 flex gap-3">

    <a
      className="inline-flex items-center
justify-center rounded-lg bg-primary px-5 py-3 text-sm
font-semibold text-primary-foreground shadow-sm"
      href="#today-goals"
    >
      これから足すものを見る
    </a>

    <a
      className="inline-flex items-center
justify-center rounded-lg border border-border
bg-background px-5 py-3 text-sm font-semibold
text-foreground"
      href="#next-step"
    >
```

読み比べ用: ここは写経しません。続けてコードを読み進めましょう。

After は `bg-card` や `text-muted-foreground` で色の役割を指定しています。実際の値は `globals.css` の変数で決まります。その変数を変えると参照している部品の色が変わります。名前から色の用途も確認できます。

```
{/* filepath: 読み比べ用サンプル(続き・実ファイルには対応  
しません) */}  
    次にやることを見る  
    </a>  
  </div>  
</section>  
);  
}  
  
export default function HomePage() {  
  return <WelcomeHero />;  
}
```

このコードの強み:

- `primary`、`card`、`accent` の名前で複数の画面に共通の色を指定できます。
- クラス名から色や面の用途を読み取れます。
- 色の変更箇所を `globals.css` にまとめられます。

覚えておきたいポイント

色や余白をファイルごとに直接書くと、共通の値を変えるたびに、複数のファイルを編集します。変数を参照すれば、その変数を定義した箇所を変えるだけで済みます。

Day 01: 開発環境を整えて初めてのアプリを動かそう

色に名前を付けるときは「主役か、補助か、背景か」という用途を考えましょう。名前と値の対応を決めてから部品で使います。

Step 5: 仕上げた画面をブラウザで見る(読む目安: 3分)

編集が終わったらもう1回ブラウザを見ます。

`npm run dev` が動いている場合は保存後の表示を確認します。

止めていたらもう一度起動します。

ターミナル(`~/workspace/task-app`)

```
npm run dev
```

開発サーバーが動いている場合は再起動不要です。止めていた場合は `Ready` がでてからブラウザを開きます。エラーが出たらターミナルに表示されたファイル名と行番号を確かめます。該当箇所を開いて貼り付け漏れやタグの対応を確認してください。

ブラウザ確認

- `http://localhost:3000` を開きます。

チェックポイント

Step 3 で `3001` などのポートを使った場合は以下の `3000` をその番号に置き換えます。

- 上に「タスク管理」と `TaskApp` が見えます。
- 「作りかけ」の小さなバッジが見えます。
- メインカードに「プロジェクトとタスクを、ひとつの画面で。」が見えます。

- ボタンが `bg-primary` の主役色で表示されています。
- `ダッシュボードへ入る` ボタンが見えます。
- 右側に「これから足すもの」「メモ」「次にやること」のカードが見えます。
- `ダッシュボードへ入る` を押すと `http://localhost:3000/dashboard` が開きます。
- `/dashboard` で「ダッシュボード」の見出しが見えます。

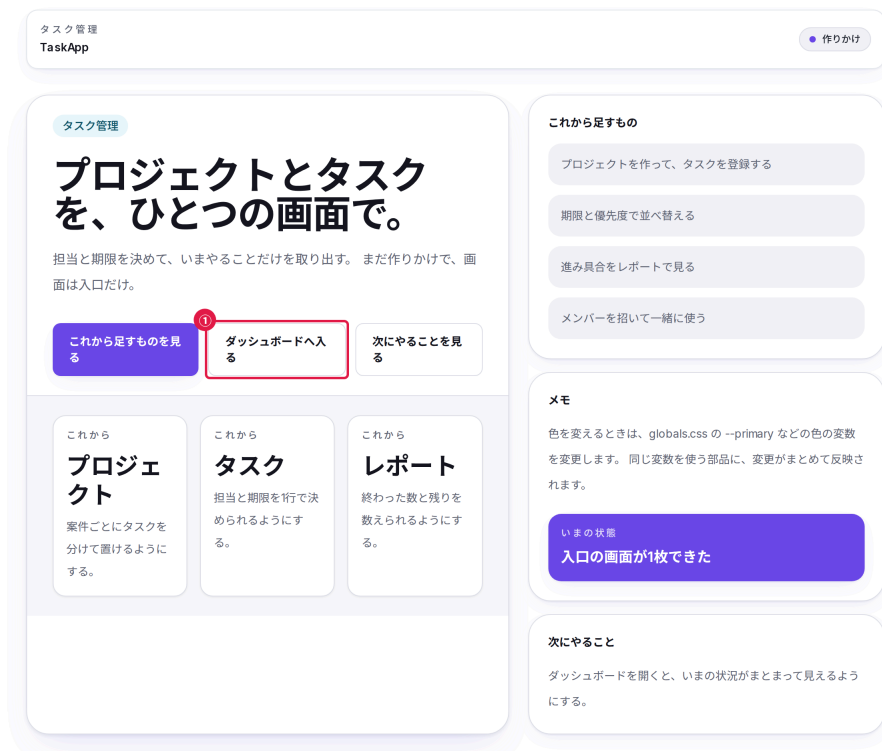


図 6: トップページ配置。上部にヘッダー、左にメインカードとボタン、右に3枚の小さなカード

赤い枠が `ダッシュボードへ入る` ボタンです。ここを押すと `/dashboard` へ移動します。

Day 01: 開発環境を整えて初めてのアプリを動かそう

冒頭のダッシュボード画像とも見比べる

今日はここまで来られたら十分です。冒頭のダッシュボード画像とも見比べてみてください。

見出しとカードの配置をチェックポイントに沿って確認します。

うまく表示されないときの見直し順

1. ターミナルにエラーが出ていないか確認します。
2. `src/app/globals.css` の貼り付け漏れがないか確認します。
3. `src/app/page.tsx` のクラス名を打ち間違えていないか確認します。
4. 開発サーバーをいったん止めて `npm run dev` で起動し直します。

今日手に入れたもの

今日は開発環境を用意してトップページとダッシュボードを作りました。

`globals.css` には部品で使う色の変数を定義しました。

明日はダッシュボードに自分のメッセージを追加します。

つまずきポイント

`Port 3000 is in use` と出て `http://localhost:3001` で起動する

原因

ポート 3000 が使用中のため、Next.js 15 が空いている番号へ移っています。

解決方法

表示された URL を開きます。以降の `3000` はその番号に読み替えてください。

Step 2 が `Cannot connect to the Docker daemon` で止まる

原因

Docker が起動していません。

解決方法

Docker Desktop を起動します。教材のコードを編集していない初期セットアップ中に限り、スクリプトを再実行してください。

Step 2 が `Permission denied` で止まる

原因

実行権限の付いていないファイルを `./scripts/...` の形で直接呼んでいます。

解決方法

`bash scripts/scaffold-from-scratch.sh` の形で実行します。`bash` にファイルを渡す書き方なら実行権限は要りません。

`bash scripts/scaffold-from-scratch.sh` の形で実行しているのに `Permission denied` で止まる

原因

書き込みのできない場所でスクリプトを動かしています。

解決方法

`pwd` で `~/workspace/task-app` にいるか確認します。違う場所なら `cd ~/workspace/task-app` してから実行し直してください。

Day 01: 開発環境を整えて初めてのアプリを動かそう

Step 4 の途中でブラウザがエラー画面になる

原因

分割されたコードの途中では、括弧やタグが閉じていません。

解決方法

「確認ポイント」を読んで次へ進みます。全ブロックを書いたあともエラーがあれば、閉じタグと貼り付け漏れを確かめてください。

`.env.example` に説明されていない行がある

原因

後の Day で使う設定も入っています。

解決方法

今日は `DATABASE_URL` を確認します。空欄の `_DEVELOPER_EMAIL` はそのままにしてください。

今日学んだ用語

用語	意味
Next.js	React でページを作るためのフレームワーク。URL とファイルの置き場所を結び付ける仕組みが最初から入っている
コンポーネント	画面の部品を関数で定義したもの。 <code>export default function Home()</code> の形で書く

用語	意味
JSX	JavaScript の中に HTML のような書き方で画面を組み立てる構文
design token	色や角丸などの値に名前を付けて1か所にまとめたもの。 <code>--primary</code> がその1つ
<code>scaffold-from-scratch.sh</code>	プロジェクトの土台一式を自動で組み立てる配布スクリプト。Day 01 で一度だけ実行する
ポート番号	1台のパソコンの中で通信の入口を区別する番号。 <code>localhost:3000</code> の <code>3000</code> がそれ
環境変数	コードの外に置く設定値。接続先やパスワードなどを <code>.env</code> に書く

応用課題: 角丸の変更が届く範囲を確かめる

見た目の値を1か所変えて影響する部品を確かめます。値を直接指定する部品と変数を参照する部品の違いを確認する課題です。

(1) `src/app/globals.css` の `:root` にある `--radius` の値をメモします。トップページを開いてヘッダー・ボタン・メインカードの角を見ておきます。

(2) `--radius` を `20px` に変えて保存します。どの部品の角が変わるか予想してからブラウザで確認します。

(3)ヘッダーとボタンの角丸が大きくなることを確認します。`rounded-[28px]` のメインカードと、`rounded-2xl` の小さなカードや箇条書きの背景は変わりません。`rounded-2xl` は Tailwind CSS が用意した `1rem` を使い、今回の `--radius` を参照しないためです。予想と違う場合は `@theme inline` の `--radius-sm` などではなく、明るいテーマの `:root` にある `--radius` を編集したか確かめます。

(4)メモした値に戻して保存します。ヘッダーとボタンの角も戻ることを確認します。

理解チェック

今日書いたコードを見ながら答えてみてください。答えは各問のすぐ下にあります。

Q1. `globals.css` の `--color-primary: hsl(var(--primary));` は何をしている行ですか。

A. `bg-primary` などのクラスと `--primary` の色を結び付ける行です。実際の色は `:root` と `.dark` の値で決まります。表示中のテーマによって使う値が切り替わります。

Q2. `:root` の `--radius: 10px;` を `20px` に変えると画面はどう変わりますか。

A. `--radius` を参照する角丸が大きくなります。`--radius-sm` から `--radius-xl` はどれもこの値が基準です。`--radius-lg` は値をそのまま使います。ほかの3つは `calc()` で増減させます。今日の画面では `rounded-xl` のヘッダーと `rounded-lg` のボタンが変わります。`rounded-[28px]` のメインカードは直接指定した値なので変わりません。`rounded-2xl` の小さなカードや箇条書きの背景も、Tailwind CSS が用意した `1rem` を使うため変わりません。

Q3.「これから足すものを見る」は `<a href="#today-goals">`、「ダッシュボードへ入る」は `<Link href="/dashboard">` と書き分けました。なぜ2つの書き方が要るのですか。

A. 移動先が違うためです。前者は同じページの `id="today-goals"` ヘスクロールします。後者は別のページへの移動です。Next.js の `Link` はページ全体を読み込み直さずに表示を切り替えるためアプリ内のページ移動に使用します。

次回予告

Day 02 では今日作った `src/app/dashboard/page.tsx` に自分の名前と今日やることを出します。

トップページはそのまま残してダッシュボードの中身だけを書き換える日です。

今日の `bg-card` や `bg-primary` は次の画面でも使います。

次に読むもの

- 次の日: Day 02^{*15}
- 全体の地図: 学びのロードマップ^{*16}
- 目次: カリキュラム目次^{*17}
- 詰まったとき: トラブルシューティング^{*18}

15. https://drive.google.com/file/d/1RraPkjcsHivFHci_b9dl5mWZBZW2aIYA/view?usp=drivesdk

16. https://drive.google.com/file/d/10py_p-6ry4RDlxBSyzPn4rtzZe74ND-T/view?usp=drivesdk

17. <https://drive.google.com/file/d/19g0cKLajEWi7s9LeWF804DB9j5kK1dt4/view?usp=drivesdk>

18. <https://drive.google.com/file/d/1OyjFWiGvZBm9QKcnKiougPZidjsn7kqR/view?usp=drivesdk>

- 言葉の意味: 用語集^{*19}

奥付

タイトル	Day 01: 開発環境を整えて初めてのアプリを動かそう
著者	磯貝光佑
版	第1版(2026年9月19日)

© 2026 磯貝光佑

本書は購入者個人の利用に限ります。本書に掲載されたコードの写経、および自分のプロジェクトへの転用・改変は自由です。本書(PDF・Markdown)および付属コードの再配布・転売・第三者との共有は、形態を問わず禁止します。

19. <https://drive.google.com/file/d/1RtElBpzkvXsoTpj4CTKPko8jnQDcPsh/view?usp=drivesdk>